Basic (guess token counts, then check) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried guessing first. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and a worked example

Hint 1 — The idea, and the exact pieces

The rule of thumb from Core Concepts is: 1 token ≈ 4 characters ≈ ¾ of a word, for plain English. To estimate a sentence, count its characters (including spaces) and divide by 4 — or count its words and multiply by roughly 1.3.

The two estimates won't always agree exactly, and that's fine — the goal isn't a precise number, it's being close enough to make a reasonable cost or context-limit decision without running code. Watch especially for: long or unusual words (these often split into 2+ tokens), and punctuation (commas and periods are usually their own token, adding a small amount you might forget to count).

The exact pieces:

- **chars ÷ 4** — fastest to compute in your head, works fine on plain English prose.
- **words × 1.3** — the alternative estimate; the two rarely land on exactly the same number, which is itself worth noticing.
- **Choose 5 sentences that deliberately span a range** — a very short one, a normal one, a long one with uncommon words, one with numbers or punctuation, and one that's mostly simple, common words.

Write down your character-based estimate and your word-based estimate separately for each sentence, before checking the real tool.

Both estimates are describing the *average* behavior of a tokenizer built by finding the most common recurring chunks in a mostly-English training set — which means the rule of thumb isn't equally wrong in both directions. It tends to *undercount* rare or long words (they split into more pieces than "1 word" suggests) and it tends to be *roughly right* on short, common, everyday words. Knowing which direction the error usually runs in is more useful than knowing the average error size, because it tells you when to pad your estimate upward on purpose rather than trust the number as-is.

There's a production-grade reason to care about getting this right beyond "will my prompt fit": **prompt caching** (previewed here, covered properly in Doc12) only reuses a cached prefix if the tokens match *exactly* — not the words, the tokens. Two prompts that look identical to a human but differ by one trailing space, one extra blank line, or a slightly different phrasing of the same instruction can tokenize differently and silently miss the cache, paying full price on every call instead of the cached, discounted rate.

Hint 2 — The plan, and a worked example

```
sentences = [
    a short one,
    a normal one,
    a long/uncommon-word one,
    a punctuation/number-heavy one,
    a plain common-words one,
]

for each sentence:
    char_guess = round(len(sentence) / 4)
    word_guess = round(word_count(sentence) * 1.3)
    write both down

for each sentence, in the tokenizer tool:
    actual = real token count
    error_chars = actual - char_guess
    error_words = actual - word_guess

compare: which estimate method was closer, and on which kind of sentence?
```

A worked example, using both estimate methods: "The cat sat on the mat." → 26 characters (including spaces) → char-based guess: 26 ÷ 4 ≈ 7. Word-based guess: 6 words × 1.3 ≈ 8. Both estimates land close together here, because this sentence is exactly the case the rule of thumb was built for — short, common, plain English words.

Now pick 4 more sentences that each break one assumption on purpose (uncommon words, heavy punctuation, a very short fragment, a longer paragraph) and repeat both estimates for each, then compare against the full Solution.

Full solution: Show me the solution